Teach Module 30: Event-Driven Architecture and Apache Kafka

Source document used only to identify the module topic: `docs/Java Software Engineer bootcamp.docx`.

Teaching note: This README teaches the topic independently and does not reuse the course material content.

Module Focus

Module 30 introduces event-driven architecture and Apache Kafka.

The main ideas are:

- Event-driven communication versus request-response communication
- Events as records of things that already happened
- Kafka topics, partitions, brokers, producers, and consumers
- Consumer groups and offset tracking
- Common use cases such as service decoupling, asynchronous processing, and audit trails

Core Explanation

In a normal request-response system, one service directly asks another service to do something.

```
Order Service -> Payment Service -> Inventory Service -> Email Service
```

The Order Service waits for each service to respond. This is simple, but it creates tight coupling. If the Payment Service is slow, the whole flow slows down. If Email Service is down, the order flow might fail unless the failure is handled carefully.

In an event-driven system, services communicate by publishing facts that happened.

```
Order Service publishes: "OrderCreated"
Payment Service listens
Inventory Service listens
Email Service listens
Analytics Service listens
```

The Order Service does not need to know who is listening. It simply says, "An order was created." Other services react independently.

The heart of event-driven architecture is:

```
A service publishes events when something important happens,
and other services respond to those events asynchronously.
```

What Is an Event?

An event is a record of something that already happened.

Good event names usually sound like past-tense facts:

```
OrderCreated
PaymentApproved
InventoryReserved
ShipmentDispatched
UserRegistered
AccountClosed
```

Bad event names often sound like commands:

```
CreateOrder
SendEmail
ChargePayment
```

An event should not tell another service what to do. It should describe what happened.

Example event:

```
{
  "eventId": "evt-1001",
  "eventType": "OrderCreated",
  "orderId": "ORD-9001",
  "customerId": "CUST-22",
  "totalAmount": 149.99,
  "createdAt": "2026-08-02T10:15:00Z"
}
```

This event says that an order exists now. Other systems can decide what to do with that information.

Where Kafka Fits

Apache Kafka is a platform for moving and storing events.

Kafka can be understood as a durable event log. Services write events into Kafka, and other services read those events when they are ready.

The main Kafka pieces are:

```
Producer -> Topic -> Consumer
```

A producer sends events.

A topic is a named stream or category of events.

A consumer reads events.

Example:

```
Order Service publishes to topic: orders
Payment Service consumes from topic: orders
Inventory Service consumes from topic: orders
```

The topic acts as the meeting place. Producers and consumers do not directly call each other.

Topics

A Kafka topic is like a channel for a type of data.

Examples:

```
orders
payments
shipments
customer-events
audit-events
```

You usually group related events in a topic. For example, the `orders` topic might contain:

```
OrderCreated
OrderCancelled
OrderUpdated
```

Or you may choose separate topics:

```
order-created
order-cancelled
order-updated
```

Both designs exist. In real systems, the choice depends on volume, ownership, schema strategy, and consumer needs.

Partitions

Kafka topics are split into partitions.

A partition is an ordered log.

```
orders topic
  partition 0: event1, event4, event7
  partition 1: event2, event5, event8
  partition 2: event3, event6, event9
```

Kafka uses partitions for scalability. More partitions allow more consumers to process messages in parallel.

Important rule:

```
Kafka preserves order inside a single partition,
not necessarily across the whole topic.
```

If ordering matters for one customer or one order, use a consistent key.

Example:

```
key = orderId
```

That ensures all events for the same order go to the same partition.

Consumer Groups

A consumer group is a team of consumers working together.

If three consumers are in the same group, Kafka divides partitions among them.

```
orders topic has 3 partitions

Payment Consumer Group:
  consumer A reads partition 0
  consumer B reads partition 1
  consumer C reads partition 2
```

This lets you scale processing.

Each consumer group gets its own independent view of the topic.

```
orders topic
  -> Payment Consumer Group
  -> Inventory Consumer Group
  -> Email Consumer Group
```

Each group tracks its own progress.

Offsets

An offset is the position of a message inside a partition.

Example:

```
partition 0:
  offset 0: OrderCreated
  offset 1: PaymentApproved
  offset 2: OrderShipped
```

Kafka consumers keep track of the offsets they have processed.

If a consumer crashes, it can restart from the last committed offset. This is how Kafka supports reliable processing.

Why Use Event-Driven Architecture?

Key benefits:

- Loose coupling: services do not need to know about each other directly.
- Scalability: consumers can process work in parallel.
- Resilience: if one consumer is down, events can remain in Kafka until it comes back.

- Extensibility: a new consumer can be added later without changing the producer.

Example:

```
Today: Payment Service listens to OrderCreated.
Later: Fraud Detection also listens to OrderCreated.
```

The Order Service does not need to change.

Trade-Offs

Event-driven systems are powerful, but they are not automatically simpler.

You must handle:

```
Duplicate messages
Out-of-order processing
Schema changes
Consumer failures
Retry behavior
Monitoring consumer lag
Event versioning
```

One of the biggest mindset shifts:

```
In event-driven systems, consistency is often eventual, not immediate.
```

Example: an order might be created now, but payment confirmation may happen seconds later.

Tiny Java Mental Model

A producer is like:

```
kafkaTemplate.send("orders", orderId, orderCreatedEvent);
```

A consumer is like:

```
@KafkaListener(topics = "orders", groupId = "payment-service")
public void handle(OrderCreatedEvent event) {
    paymentService.startPayment(event);
}
```

Spring Kafka details belong more naturally in Module 31. For Module 30, focus on the architecture and vocabulary.

Check Your Understanding

Answer these in your own words:

1. What is the difference between a command and an event?
2. Why does Kafka use topics?

3. Why are partitions important?
4. What problem do consumer groups solve?
5. What does an offset represent?

Practice Exercises

Exercise 1: Identify Events vs Commands

Classify each item as an event or a command:

```
OrderCreated
CreateOrder
PaymentFailed
SendWelcomeEmail
CustomerRegistered
ReserveInventory
InventoryReserved
ShipmentDispatched
```

Then rewrite the commands as events where possible.

Example:

```
SendWelcomeEmail -> UserRegistered
```

Exercise 2: Model an Event Flow

Design an event-driven flow for an online order system.

Services:

```
Order Service
Payment Service
Inventory Service
Notification Service
Shipping Service
```

Use events like:

```
OrderCreated
PaymentApproved
InventoryReserved
ShipmentRequested
ShipmentCreated
```

Goal: show which service publishes each event and which service consumes it.

Exercise 3: Design Kafka Topics

For an e-commerce system, decide whether you would use:

```
orders
payments
inventory
notifications
```

or more specific topics like:

```
order-created
payment-approved
inventory-reserved
```

For each choice, explain the trade-off.

Exercise 4: Choose Event Keys

Pick a Kafka message key for each event:

```
OrderCreated
PaymentApproved
CustomerUpdated
InventoryAdjusted
ShipmentCreated
```

Example:

```
OrderCreated -> orderId
```

Then explain why the key matters for ordering and partition assignment.

Exercise 5: Partition Reasoning

Suppose the `orders` topic has 4 partitions.

Events with the same `orderId` always go to the same partition.

Answer:

```
Why is this useful?
What ordering guarantee does Kafka provide?
Can Kafka guarantee total ordering across all 4 partitions?
```

Exercise 6: Consumer Group Scenario

You have a topic called `orders` with 6 partitions.

Payment Service has one consumer group with 3 instances.

Answer:

```
How many partitions can each instance process?
What happens if you scale Payment Service to 6 instances?
What happens if you scale it to 8 instances?
```

Key idea: consumers in the same group share work, but a partition can only be actively consumed by one consumer in that group at a time.

Exercise 7: Offset Recovery

A consumer reads these records:

```
offset 10: OrderCreated
offset 11: PaymentApproved
offset 12: InventoryReserved
```

It processes offset 10 and 11, then crashes before committing offset 12.

Answer:

```
What happens when it restarts?
Why can this lead to duplicate processing?
How would you make the consumer safer?
```

Exercise 8: Duplicate Message Handling

Design an idempotent consumer for this event:

```
{
  "eventId": "evt-1001",
  "eventType": "PaymentApproved",
  "orderId": "ORD-500"
}
```

Question:

```
How can the consumer avoid processing the same payment event twice?
```

Hint: store processed `eventId`s or enforce unique business records.

Exercise 9: Event Schema Design

Create a JSON event for:

```
CustomerRegistered
```

Include:

```
eventId
eventType
```

```
customerId
email
registeredAt
source
version
```

Then explain why `version` is useful.

Exercise 10: Event-Driven vs Request-Response

For each case, choose Kafka/event-driven or direct REST request-response:

```
User logs in and needs immediate authentication result
Order is created and email confirmation should be sent
Payment must be approved before checkout completes
Audit trail must record account changes
Analytics system needs order data
Frontend needs to fetch current customer profile
```

Explain your reasoning.

Mini Project Practice

Build a small design, no code required at first:

```
Order Service publishes OrderCreated
Payment Service consumes OrderCreated and publishes PaymentApproved or PaymentFailed
Inventory Service consumes PaymentApproved and publishes InventoryReserved
Notification Service consumes OrderCreated, PaymentApproved, and PaymentFailed
```

For each service, define:

```
Input events
Output events
Kafka topics
Message keys
Failure handling
```

Module 30 Lab: Event-Driven Architecture with Kafka

Goal

Design and reason through a Kafka-based event-driven order workflow.

You do not need Spring Boot yet. This lab is mostly architecture thinking.

Scenario

You are building a simple online shopping system with these services:

```
Order Service
Payment Service
Inventory Service
Notification Service
Audit Service
```

When a customer places an order, the system should process payment, reserve inventory, notify the customer, and record an audit trail.

Part 1: Identify Events

Create events for the workflow.

Use past-tense names:

```
OrderCreated
PaymentApproved
PaymentFailed
InventoryReserved
InventoryUnavailable
CustomerNotified
```

Avoid command-style names like:

```
ProcessPayment
SendEmail
ReserveInventory
```

Task: write 6 to 8 events your system needs.

Example:

```
OrderCreated
PaymentApproved
PaymentFailed
InventoryReserved
InventoryRejected
OrderCompleted
OrderCancelled
NotificationSent
```

Part 2: Design Event Payloads

Create a JSON payload for `OrderCreated`.

Example:

```
{
  "eventId": "evt-001",
  "eventType": "OrderCreated",
```

```
  "orderId": "ORD-1001",
  "customerId": "CUST-501",
  "items": [
    {
      "productId": "PROD-10",
      "quantity": 2
    }
  ],
  "totalAmount": 129.99,
  "createdAt": "2026-08-02T14:30:00Z",
  "version": 1
}
```

Now create payloads for:

```
PaymentApproved
InventoryReserved
PaymentFailed
```

Part 3: Choose Kafka Topics

Create topic names for your system.

Simple option:

```
orders
payments
inventory
notifications
audit
```

More specific option:

```
order-events
payment-events
inventory-events
notification-events
audit-events
```

Task: choose your topic design and explain why.

Example answer:

```
I will use order-events, payment-events, and inventory-events because each topic maps to a
business domain. This keeps related events together while avoiding too many tiny topics.
```

Part 4: Map Producers and Consumers

Fill this table:

Service	Consumes Events	Publishes Events
Order Service	None / API request	OrderCreated
Payment Service	OrderCreated	PaymentApproved, PaymentFailed
Inventory Service	PaymentApproved	InventoryReserved, InventoryUnavailable
Notification Service	OrderCreated, PaymentApproved, PaymentFailed	CustomerNotified
Audit Service	All events	None

Part 5: Pick Message Keys

Choose the Kafka message key for each event.

Recommended:

```
key = orderId
```

Example:

Event	Kafka Key	Why
OrderCreated	orderId	Keeps all events for one order in order
PaymentApproved	orderId	Payment belongs to one order
InventoryReserved	orderId	Inventory reservation follows order flow

Part 6: Consumer Groups

Define consumer groups:

```
payment-service-group
inventory-service-group
notification-service-group
audit-service-group
```

Answer these:

```
If Payment Service has 3 running instances, should they use the same groupId?
If Notification Service and Audit Service both read OrderCreated, should they use the same
groupId?
Why or why not?
```

Expected idea:

```
Payment Service instances should share one groupId so they split the work.
Notification and Audit should use different groupIds so both receive every event
```

```
independently.
```

Part 7: Failure Handling

For each failure, decide what should happen:

```
Payment Service is temporarily down
Inventory Service receives duplicate PaymentApproved event
Notification Service fails to send email
Audit Service falls behind by 10,000 messages
```

Use these ideas:

```
Retry later
Store processed event IDs
Move bad messages to a dead-letter topic
Monitor consumer lag
Make consumers idempotent
```

Part 8: Draw the Flow

Write the final flow like this:

```
1. Customer places order.
2. Order Service publishes OrderCreated to order-events.
3. Payment Service consumes OrderCreated.
4. Payment Service publishes PaymentApproved to payment-events.
5. Inventory Service consumes PaymentApproved.
6. Inventory Service publishes InventoryReserved to inventory-events.
7. Notification Service consumes OrderCreated and PaymentApproved.
8. Audit Service consumes all major events.
```

Submission Checklist

Create a short lab answer with:

```
1. Event list
2. Event JSON payloads
3. Topic names
4. Producer/consumer table
5. Message keys
6. Consumer groups
7. Failure-handling strategy
8. Final event flow
```

Instructor review guidance:

```
When reviewing a completed lab, check correctness, event naming, topic design,
```

```
consumer group reasoning, offset awareness, idempotency, and failure handling.
```